1. RESTful API Nedir?

REST (Representational State Transfer), web hizmetleri oluşturmak için kullanılan bir mimari stildir. RESTful API ise bu kurallara uyan bir API'dir. Spring Boot ile Controller katmanını genellikle bir RESTful API sunmak için kullanırız.

Bilmen gereken temel prensipler:

- **İstemci-Sunucu (Client-Server):** İstemci (örn. React uygulaması veya mobil uygulama) sadece arayüzle ilgilenir, sunucu (Spring Boot) ise veri erişimi ve iş mantığıyla.
- **Durumsuzluk (Stateless):** Sunucu, istemcinin önceki durumunu veya oturumunu hatırlamaz. İstemciden gelen her istek, sunucunun o isteği anlaması ve işlemesi için gereken *tüm* bilgiyi içermelidir (örn. Authorization token).
- **Kaynak Odaklılık (Resource-Based):** Her şey bir "kaynak" (resource) olarak düşünülür (Örn: Öğrenciler, Ürünler, Siparişler). Bu kaynaklara URL'ler (URI) aracılığıyla erişilir (örn. api/v1/products).
- **Standart Format:** Veriler genellikle **JSON** (JavaScript Object Notation) formatında alınıp verilir.

---

2. @RestController Annotation

Sınıfın bir REST API uç noktası (endpoint) olduğunu Spring'e bildiren en temel anotasyondur. Sınıfın en üstüne yazılır.

Ne işe yarar? Aslında @Controller ve @ResponseBody anotasyonlarının birleşimidir.

- @Controller: Bu sınıfın bir Spring MVC Controller'ı olduğunu belirtir.
- @ResponseBody: Bu sınıftaki metodların döndürdüğü değerlerin bir web sayfasına (HTML) değil, **doğrudan HTTP yanıtının gövdesine (body)** yazılacağını belirtir. Spring Boot arka planda Jackson kütüphanesini kullanarak döndürdüğün Java nesnelerini otomatik olarak JSON formatına çevirir.

Java

```
@RestController

@RequestMapping("/api/users") // Sınıf seviyesinde temel URL'i belirler

public class UserController {

   // Metodlar buraya gelecek

}
```

---

3. HTTP Metodları ve Annotations

RESTful mimaride CRUD (Create, Read, Update, Delete) işlemlerini yapmak için HTTP metodlarını kullanırız. Spring Boot, bu metodları yakalamak için özel anotasyonlar sunar.

- **@GetMapping (Read / Okuma):**
  - Sunucudan veri çekmek için kullanılır. Veritabanında bir değişiklik yapmaz.
  - Örnek: Tüm kullanıcıları listelemek.
- **@PostMapping (Create / Oluşturma):**
  - Sunucuya yeni bir veri/kaynak eklemek için kullanılır. Gönderilen veriler genellikle isteğin gövdesinde (@RequestBody ile) yer alır.
  - Örnek: Yeni bir kullanıcı kaydetmek.
- **@PutMapping (Update / Güncelleme):**
  - Var olan bir kaynağı tamamen güncellemek (değiştirmek) için kullanılır.
  - Örnek: Bir kullanıcının tüm bilgilerini değiştirmek.
- **@DeleteMapping (Delete / Silme):**
  - Var olan bir kaynağı silmek için kullanılır.

Örnek Kullanımlar:

Java

```
@GetMapping
public List<User> getAllUsers() { ... }


@PostMapping
public User createUser(@RequestBody User newUser) { ... }


@PutMapping("/{id}")
public User updateUser(@PathVariable Long id, @RequestBody User updatedUser) { ... }


@DeleteMapping("/{id}")
public void deleteUser(@PathVariable Long id) { ... }
```

---

4. @PathVariable Kullanımı

İstekte bulunulan URL'in **yolundaki (path)** dinamik değerleri yakalamak için kullanılır. Genellikle belirli bir kaynağı ID'sine göre getirmek, güncellemek veya silmek istediğimizde tercih ederiz.

- URL Örneği: /api/users/5 (Buradaki 5 bir değişkendir).

Java

```
// Süslü parantez içindeki isim ile değişkendeki isim aynı olmalıdır.
@GetMapping("/{userId}")
public User getUserById(@PathVariable("userId") Long id) {
  // id değişkeni 5 değerini alır.
  return userService.findById(id);
}
```

---

5. @RequestParam Kullanımı

URL'in sonuna eklenen **sorgu parametrelerini (query parameters)** yakalamak için kullanılır. Filtreleme, arama veya sayfalama (pagination) işlemleri için biçilmiş kaftandır.

- URL Örneği: /api/users?status=active&age=25 (? işaretinden sonraki kısımlar).

Java

```
@GetMapping("/search")
public List<User> searchUsers(
    @RequestParam(name = "status", required = false, defaultValue = "all") String status,
    @RequestParam(name = "age") Integer age) {

  // required = false: Bu parametrenin gönderilmesi zorunlu değildir.
  // defaultValue = "all": Eğer status gönderilmezse varsayılan olarak "all" değerini al.
  return userService.search(status, age);
}
```

Kısa Özet: Ne Zaman Hangisi?

- Belirli bir kaynağı tanımlıyorsan (Örn: 10 numaralı sipariş): @PathVariable kullan. -> /orders/10
- Kaynakları filtrelik veya sıralıyorsan: @RequestParam kullan. -> /orders?status=shipped

---

6. ResponseEntity Sınıfı

Bir API yazarken sadece veriyi (User, String, List vb.) döndürmek çoğu zaman yeterli ve profesyonel değildir. İstek sonucunda doğru **HTTP Status Kodunu** (Örn: 200 OK, 201 Created, 404 Not Found) ve gerekirse özel **HTTP Header**'larını (Başlıklarını) döndürmen gerekir. İşte ResponseEntity bunu yapmanı sağlar. Tüm HTTP yanıtını (Status, Header, Body) temsil eder.

Neden Kesinlikle Kullanmalısın? Eğer sadece return user; dersen, Spring Boot her zaman varsayılan olarak 200 OK döndürür. Ancak yeni bir kayıt oluşturulduğunda 201 Created veya aranan kayıt bulunamadığında 404 Not Found dönmek REST standartları için çok önemlidir.

Örnek Kullanımlar:

Başarılı bir dönüş (200 OK):

Java

```
@GetMapping("/{id}")
public ResponseEntity<User> getUser(@PathVariable Long id) {
  User user = userService.findById(id);
  return ResponseEntity.ok(user); // Status: 200, Body: user
}
```

Yeni Kayıt Oluşturma (201 Created):

Java

```
@PostMapping
public ResponseEntity<User> createUser(@RequestBody User user) {
  User savedUser = userService.save(user);
  return new ResponseEntity<>(savedUser, HttpStatus.CREATED); // Status: 201
}
```

Hata Durumu / Bulunamadı (404 Not Found):

Java

```
@GetMapping("/{id}")
public ResponseEntity<User> getUserWithCheck(@PathVariable Long id) {
  User user = userService.findById(id);
  if (user == null) {
    return ResponseEntity.notFound().build(); // Status: 404, Body: Boş
  }
  return ResponseEntity.ok(user);
}
```

Controller katmanının temel taşları bunlardır. Bu yapıları kullanarak sağlam ve standartlara uygun bir API inşa edebilirsin.

*RestEmployeeController.java × EmployeeService.java EmployeeRepository.java SpringApplication.class

```
1  package com.enesbayram.controller;
2
3  import java.util.List;
14
15 @RestController
16 @RequestMapping("/rest/api/employee")
17 public class RestEmployeeController {
18
19     @Autowired
20     private EmployeeService employeeService;
21
22     @GetMapping(path = "/employee-list")
23     public List<Employee> getAllEmployeeList(){
24         return employeeService.getAllEmployeeList();
25     }
26
27     @GetMapping(path = "/employee-list/{id}")
28     public Employee getEmployeeById(@PathVariable(name = "id", required = true) String id) {
29         return employeeService.getEmployeeById(id);
30     }
31
32
33     @GetMapping(path = "employee-list-with-params")
34     public List<Employee> getEmployeeWithParams(@RequestParam(name = "firstName") String firstName,
35                                                 @RequestParam(name = "lastName") String lastName){
36
37         System.out.println(firstName + " " + lastName);
38         return null;
```

RestEmployeeController.java | EmployeeService.java | EmployeeRepository.java | SpringApplication.class

```java
package com.enesbayram.services;

import java.util.List;

@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository employeeRepository;


    public List<Employee> getAllEmployeeList(){
        return employeeRepository.getAllEmployeeList();
    }

    public Employee getEmployeeById(String id) {
        return employeeRepository.getEmployeeById(id);
    }

    public List<Employee> getEmployeeWithParams(String firstName , String lastName){

        return null;
    }

}
```

SPRİNG DERSLERİ / @RequestParam

GET http://localhost:8080/rest/api/employee/with-params?firstName=Enes

Params | Authorization | Headers (7) | Body | Scripts | Tests | Settings

Query Params

Key	Value
firstName	Enes
lastName	Bayram
Key	Value

Body | Cookies | Headers (5) | Test Results

Pretty | Raw | Preview | Visualize | JSON


```
[
    {
        "id": "1",
        "firstName": "Enes",
        "lastName": "Bayram"
    },
    {
        "id": "6",
        "firstName": "Enes",
        "lastName": "Yalçın"
    }
]
```

SPRİNG DERSLERİ / **@RequestParam**

GET http://localhost:8080/rest/api/employee/with-params

Params • Authorization Headers (7) Body Scripts Tests Settings

Query Params

Key	Value	D
firstName	Enes	
lastName	Bayram	
Key	Value	D

Body Cookies Headers (5) Test Results 2

Pretty Raw Preview Visualize JSON


```
[
    {
        "id": "1",
        "firstName": "Enes",
        "lastName": "Bayram"
    },
    {
        "id": "2",
        "firstName": "Yakup",
        "lastName": "Reçber"
    },
    {
        "id": "3",
        "firstName": "Bilal",
        "lastName": "Çamur"
```

SPRİNG DERSLERİ | GET @RequestParam

SPRİNG DERSLERİ / **@RequestParam**

GET http://localhost:8080/rest/api/employee/with-params?firstName=Enes&lastName=Bayram

Params Authorization Headers (7) Body Scripts Tests Settings

Query Params

	Key	Value
☑	firstName	Enes
☑	lastName	Bayram
	Key	Value

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON


```
[
    {
        "id": "1",
        "firstName": "Enes",
        "lastName": "Bayram"
    }
]
```